#A The ConfidentialTokenStore marker interface should go in ConfidentialTokenStore.java
#B The AuthenticatedTokenStore should go in AuthenticatedTokenStore.java
#C The SecureTokenStore combines them and goes in SecureTokenStore.java

You can now change each of the token stores to implement an appropriate interface:

- If you assume that the backend cookie storage is secure against injection and other attacks, then the `CookieTokenStore` can be updated to implement the `SecureTokenStore` interface.
- If you've followed the hardening advice from chapter 5, the `DatabaseTokenStore` can also be marked as a `SecureTokenStore`. If you want to ensure that it is always used with HMAC for extra protection against tampering, then mark it as only confidential.
- The `JsonTokenStore` is completely insecure on its own, so leave it implementing the base `TokenStore` interface.
- The `SignedJwtTokenStore` provides no confidentiality for claims in the JWT, so it should only implement the `AuthenticatedTokenStore` interface.
- The `HmacTokenStore` turns any `TokenStore` into an `AuthenticatedTokenStore`. But if the underlying store is already confidential then the result is a `SecureTokenStore`. You can reflect this difference in code by making the `HmacTokenStore` constructor private and providing two static factory methods instead, as shown in listing 6.10. If the underlying store is confidential then the first method will return a `SecureTokenStore`. For anything